

Project TODO

This is the single current, forward-looking checklist, refreshed against the repository on 2026-09-09. Completed history is summarized at the end instead of being left in the active sequence. Work in order: later modules depend on the decisions and contracts established earlier.

☐ 1. Accept the event-loop causality ADR before implementing engine modules.

The ADR must describe one deterministic sequence from a market event entering the engine to observation, strategy decisions, order arrival, fills and accounting. It must use the domain terms in `CONTEXT.md` consistently.

Already decided:

- A successfully decoded and accepted event mutates `OrderBook` before the strategy sees it.
- Every successfully processed event reaches the strategy's observation path, including events that leave the market not decision-ready.
- Observation never grants execution authority. A strategy does not call `ExecutionVenue` directly; it produces `OrderIntent` values only when the engine requests a decision.
- Book trust and market shape are independent facts: `BookHealth::isTrusted()` reports snapshot/stream trust, while `OrderBook::marketShape()` reports `empty`, `one_sided`, `locked`, `crossed` or `open`.
- Market shape is evaluated only at a declared safe checkpoint after classification, reconciliation and book mutation. An intermediate crossed state is not diagnosed early.
- One engine-owned `DecisionGate` combines book trust, market shape and operational trading state. Strategies must not reproduce this policy independently.
- Every `OrderIntent` must also pass a centralized `RiskGate`; `ExecutionVenue` performs a final defensive check and every rejection has a named audit reason.
- Full `OrderBook::validateStructure()` runs after successful mutations in debug builds. Release replay uses explicit checkpoints; production hot paths require cheap local checks rather than a full-book scan per event.

Still to decide through concrete scenarios:

1. **[Resolved 2026-09-13]** The exact minimal Stage 5 replay operational states and transitions. Live pause/resume, shutdown orchestration and kill-switch wiring remain Stage 9 work — `OperationalState` is deferred whole to Stage 9, since at Stage 5 it would have exactly one unvarying value and nothing to test.
2. When a submitted intent enters the simulated outbound-latency queue.
3. Whether an order can first fill at its arrival timestamp or only on later market activity.
4. How exchange, receipt, simulation and equal timestamps are ordered deterministically.
5. The exact sequence for fill acknowledgement, fees, cash, position, average price and PnL.
6. What observation and decision callbacks occur while unseeded, synchronizing, corrupted, disconnected, gapped or resynchronizing.
7. Which decision-block and order-rejection reasons are permanent public contracts.
8. When a crossed safe checkpoint becomes persistent enough to mark book trust corrupted and request resynchronization; do not invent a threshold without evidence.

Done when: one accepted ADR answers all eight plan-v4 causality questions, states what is deliberately deferred, includes at least one hand-worked event timeline, and defines a deterministic reason for every blocked decision or rejected order.

☐ 2. Implement exact-integer portfolio and accounting behaviour test-first.

Define cash, signed position, average entry price, realized PnL, unrealized PnL and fees using exact integer units — never `float` or `double`. Start with a hand-calculated sequence containing a buy, partial exit, final exit and fees before designing the class around it.

Done when: isolated tests reproduce every hand calculation exactly, distinguish realized from unrealized PnL, cover long/flat/short transitions, and reject arithmetic overflow without partially changing state.

☐ 3. Define order intentions, decision status and reason taxonomies.

Add the smallest value types required by the accepted ADR: `OrderIntent`, operational trading state, decision-block reason and order-rejection reason. Keep observation, permission to decide and permission to execute distinct. Do not add an interface until a real caller needs it.

Done when: unit tests demonstrate that untrusted data, every non-open market shape, paused operation and kill-switch activation produce distinct deterministic reasons.

☐ 4. Add the `ExecutionVenue` seam, minimal admission/risk policy and deterministic simulated adapter.

Begin with submit, accept/reject, fill and cancel behaviour. The risk gate checks quantity, notional and resulting-position limits with real behaviour and rejection tests. The simulated venue independently rechecks safety before acceptance and records a reasoned audit result. Implement the no-latency behaviour first; add the latency queue only according to the ADR. Order-rate, drawdown, production kill-switch and live recovery controls stay explicitly deferred to Stage 9 — do not create `return true` placeholders for them.

Done when: submit/accept/reject/fill/cancel scenarios are directly unit-testable, rejection leaves portfolio and open-order state unchanged, and strategies cannot bypass the gate by calling the venue directly.

☐ **5. Add the Strategy seam and NoopStrategy.**

Separate unconditional ordered observation from gated decision-making. A strategy receives a lifetime-bounded read-only `BookView` and returns `OrderIntent` values; it receives neither a mutable `OrderBook` nor direct venue authority.

Done when: NoopStrategy observes every supplied event, produces no intentions, compiles against the real interface, and is directly testable without an engine loop.

☐ **6. Implement the single-threaded deterministic engine loop.**

Inject `Feed`, `Clock`, `Strategy`, `DecisionGate`, the minimal admission/risk policy, `ExecutionVenue`, `OrderBook`, `BookHealth` and `Portfolio`. Implement the exact ordering accepted in the ADR before adding concurrency or performance optimization.

Done when: a synthetic timeline proves apply-before-observe, observation during blocked states, gated decisions, deterministic latency/arrival ordering and exact post-fill accounting.

☐ **7. Pass the Stage 5 evidence gates.**

- A no-op strategy conserves event counts, cash, position and PnL over a committed capture.
- One scripted strategy produces a real intention and the hand-calculated accept/fill/fee/PnL scenario matches exactly end to end.
- At least one real admission rule rejects an intention without changing venue or portfolio state; a no-op `RiskGate` does not satisfy this gate.
- Ten identical runs produce identical event, book, decision, order, cash, position and PnL digests.
- Every blocked decision and rejected order is counted by its named reason.

Done when: all gates pass from a clean checkout without relying on private capture files.

☐ **8. Ship the replay executable and refresh current documentation.**

Wire `apps/replay_main.cpp` into CMake. It must run the committed capture through the real engine with `NoopStrategy` and print a concise report covering provenance, counts, health, market-shape reasons, decisions, orders, accounting and deterministic digests. Then update plan v4, README, `docs/handoff/status.md` and placeholder module comments to describe what actually exists.

Done when: cmake --build produces the replay executable, its report is reproducible, and no current document labels implemented modules as placeholders.

Closed foundation

- ✓ Capture decoding uses one `decodeCapturedOrder` pass while the legacy order-only decoder remains available to the old recorder path.
- ✓ Clock and floating-point architecture guards are active in CI and have deliberately broken tests.
- ✓ `OrderBook` now separates debug structural validation from reason-coded market shape.
- ✓ `BookHealth::isTrusted()` and `OrderBook::marketShape()` have distinct meanings documented in `CONTEXT.md`.
- ✓ `OrderBook::apply()` dispatches by event kind, preserves failure atomicity for ordinary rejected mutations, and runs structural validation after successful debug-build mutations.
- ✓ The full suite passed 303 tests after the trust/shape interface migration on 2026-09-09.

See *CLAUDE.md* for the working agreement this list follows. Refresh this checklist when a step closes or when reality disagrees with it — a stale checklist is worse than none.